Universidad Mariano Gálvez de Guatemala
Sede Antigua Guatemala
Facultad de Ingeniería en Sistemas de Información
Y Ciencias de la Computación
Sección "A"



Documentacion

Karen Beatriz Canú Cocon - 0910-23-10072

Presentación y alcance documental

BecaNet centraliza el proceso académico de gestión de becas desde la convocatoria y la postulación hasta la evaluación, la decisión de coordinación y el seguimiento. Este documento reúne y revisa los anexos, el diseño, los planes de calidad y pruebas, el control de patrones y el manual técnico en una sola entrega coherente con la modalidad individual.

La referencia institucional Ministerio de Educación corresponde al caso de estudio definido por el curso. No se presenta el sistema como una plataforma oficial contratada, implantada o aceptada por esa institución.

`https://github.com/karencz0909/Proyecto_individual-_AnalisisSistemas`

Contenido

Resumen del sistema y reglas de negocio

Anexo 1 Constitución y organización individual

Anexo 2 Revisión de Sprint y registro de evidencia

Anexo 3 Política de trabajo individual

Anexo 4 Visión del producto y Product Backlog

Anexo 5 Arquitectura y diseño del sistema

Anexo 6 Plan de aseguramiento de la calidad

Anexo 7 Plan de pruebas

Anexo 8 Patrones de diseño y control de objetivos

Anexo 9 Manual técnico

Anexo 10 Guía de uso por rol

Resumen del sistema y reglas de negocio

Problema y propósito

El enunciado académico plantea un proceso de becas disperso entre formularios, hojas de cálculo y correo electrónico, con riesgo de duplicidad, pérdida documental, retrasos y poca trazabilidad. BecaNet propone concentrar los expedientes y separar las responsabilidades de postulación, evaluación y resolución.

Tecnologías y capas

Componente	Tecnología y función
Presentación	Angular 18.2 y TypeScript; aplicación SPA con HTML y CSS.
API y negocio	ASP.NET Core 8 Web API y C#; controladores, servicios y DTO.
Persistencia	Entity Framework Core 8, repositorios y Microsoft SQL Server.
Autenticación	JWT Bearer; control por rol en las rutas y operaciones que lo declaran.
Documentación API	Swagger en Development.
Archivos	PDF, JPG, JPEG y PNG; hasta 5 × 1024 × 1024 bytes; wwwroot/uploads.
Pruebas	Backend net9.0 con NUnit y xUnit, Moq e InMemory; frontend Karma y Jasmine.

Responsabilidades funcionales

Rol	Operaciones previstas
Estudiante	Registro, consulta de convocatorias abiertas, solicitud con motivo y monto, carga documental, cancelación permitida y consulta de avance y notificaciones.
Evaluador	Consulta de expedientes asignados, revisión documental, puntaje de 0 a 100 y observaciones; finalización de revisión. No decide la aprobación.
Coordinador	Convocatorias y perfiles, comités y asignaciones, decisiones por monto, auditoría y reportes. Algunas funciones están expuestas por API y requieren completar o verificar su interfaz.

Reglas de negocio consolidadas

RN01 La postulación se admite a convocatorias en estado ABIERTA. Las fechas se almacenan; no se presume que el cierre por fecha sea automático.

RN02 Se rechaza una segunda solicitud activa de un estudiante para la misma convocatoria. El índice único del DbContext cubre todas las solicitudes del par estudiante y convocatoria: debe revisarse la posibilidad de repostular tras cancelar.

RN03 Los archivos deben tener extensión permitida y tamaño máximo de 5 MiB, expresado como 5 MB en la interfaz. El código comprueba extensión y tamaño; no acredita inspección antivirus ni validación completa del contenido.

RN04 Un comité necesita al menos dos evaluadores. La distribución automática omite expedientes sin certificación de estudios.

RN05 La calificación pertenece al evaluador asignado y se mantiene separada de la decisión administrativa.

RN06 Hasta Q10,000 inclusive, un coordinador resuelve una solicitud EVALUADA. Para más de Q10,000 se exige revisión REVISADO, preaprobación y decisión final de otro coordinador, con verificación de identificador y correo.

RN07 Una preaprobación favorable no modifica el estado principal a APROBADA. Un rechazo requiere motivo y detiene el camino a aprobación.

RN08 Cancelación, decisiones y otras operaciones relevantes incorporan registros de auditoría y notificación según el servicio. No se presume notificación por correo o WhatsApp.

Límites de la versión documentada

No se acreditan desembolsos, integración con entidades externas, correo real, exportación a PDF o Excel, despliegue productivo, cumplimiento normativo certificado ni capacidad nacional de operación. El modo claro y oscuro es una función de interfaz; su accesibilidad y compatibilidad requieren pruebas registradas.

Anexo 1 Constitución y organización individual

Adaptación del Documento de Constitución del Equipo a una única responsable. Se conserva la organización solicitada en el PDF y se elimina la sección adicional de selección de nombre que desplazaba la numeración original.

1 Identificación del proyecto

Campo	Dato
Nombre	BecaNet
Proyecto asignado	Plataforma Nacional para la Gestión Integral de Becas — Ministerio de Educación
Modalidad	Individual, según declaración de la autora
Curso	Análisis de Sistemas II
Product Owner académico	Ing. Mario Fuentes, docente según el enunciado

2 Integrante responsable

Nombre	Rol funcional	Contacto
Karen Beatriz Canú Cocon	Análisis, arquitectura, desarrollo, pruebas y documentación	kcanuc@miumg.edu.gt Teléfono no proporcionado

3 Objetivo general y objetivos específicos

Objetivo general: diseñar e implementar individualmente un MVP de BecaNet que integre el ciclo de gestión de becas mediante una aplicación web con roles diferenciados, persistencia relacional y trazabilidad de decisiones, aplicando y justificando las decisiones de ingeniería de software del curso.

Objetivo específico 1: integrar para el 28 de septiembre de 2026 el acceso, registro, perfiles y convocatorias con solicitudes, documentos, comités y evaluación. Criterio verificable: recorrido con cuentas de los tres roles y evidencia asociada a las historias correspondientes.

Objetivo específico 2: completar para el 12 de octubre de 2026 la resolución y los reportes básicos, y registrar pruebas unitarias e integración de las reglas críticas. Criterio verificable: casos de aprobación simple, doble coordinación, rechazo y conteos contrastados con expedientes de prueba.

Objetivo específico 3: consolidar para el 26 de octubre de 2026 los anexos, manuales, justificación de arquitectura y patrones y plan de pruebas. Criterio verificable: documentación consistente con la versión identificada del repositorio y pendientes explícitos antes de la entrega oficial del 31 de octubre.

Las fechas provienen del Anexo 1 original. Estos objetivos describen metas históricas; este documento no certifica su cumplimiento en esas fechas.

5 Protocolo de comunicación única

ClickUp, espacio Análisis - Becas, se consigna en los documentos aportados como medio para organizar tareas; su contenido no fue consultado. El canal de comunicación con el docente no está identificado. Como protocolo propuesto, cada consulta debe incluir historia o requisito, evidencia, decisión solicitada y fecha; las respuestas deben vincularse al backlog y al commit afectado.

6 Cronograma del proyecto WBS

5.1. Resumen general

Dato	**Valor**
Inicio real del proyecto	sábado 8 de agosto de 2026
Fecha de este corte (cambio a individual)	miércoles 16 de septiembre de 2026
Nueva fecha límite personal	lunes 26 de octubre de 2026
Fecha oficial de entrega (según fecha entrega)	31 de octubre de 2026
Margen entre entrega personal y fecha oficial	5 días (27 al 31 de octubre)

5.2. Trabajo ya completado (08/08 – 15/09/2026)

- Selección del nombre del proyecto, identificación y objetivos iniciales.
- Documento de Visión del Producto y Product Backlog inicial: 6 Épicas y 17 Historias de Usuario con criterios de aceptación (Anexo 4).
- Selección y justificación de la arquitectura de software (En Capas) y del stack tecnológico (Angular, ASP.NET Core, Entity Framework Core, SQL Server, JWT).
- Diagrama de Casos de Uso, Diagrama de Componentes y Diagrama de Clases (modelo de dominio).
- Diseño de la Base de Datos (modelo relacional) y script T-SQL ejecutable para SQL Server (11 tablas).
- Documento de Diseño de Sistema Evolutivo (Anexo 5) con la justificación de la arquitectura.
- Código backend (ASP.NET Core Web API) de los módulos de Solicitudes, Documentación, Comités Evaluadores y Evaluaciones.
- Código frontend (Angular) de esos mismos 4 módulos, con sus pantallas, servicios y modelos ya compilados sin errores.

5.3. Plan restante (16/09 – 26/10/2026)

FASE 1 — Módulos Base (16/09 – 21/09/2026)

Tarea	**Fecha**	**Comentarios**
Backend: módulo de autenticación (registro/login)	18/09/2026	Incluye validaciones de los criterios de aceptación ya definidos

Backend: módulo de gestión de perfil de usuario	19/09/2026	Con pruebas unitarias asociadas
Backend: módulo de administración de convocatorias (crear/publicar/cerrar)	20/09/2026	Corresponde al Objetivo Específico 1
Frontend: pantallas de registro, login, perfil y convocatorias	21/09/2026	Conectadas a los endpoints anteriores

FASE 2 — Integración y Patrones (22/09 – 28/09/2026)

Tarea	Fecha	Comentarios
Integrar módulos base con los ya construidos (Solicitudes, Documentos, Comités, Evaluaciones)	24/09/2026	Prueba manual del flujo completo de principio a fin
Redactar la Justificación de Patrones de Diseño (2 creacionales, 2 estructurales, 2 de comportamiento)	27/09/2026	Debe responder qué problema resuelve cada patrón
Hito: Objetivo Específico 1 cumplido	28/09/2026	Sistema base + Sprint 3 integrados y funcionando

FASE 3 — Aprobación y Reportes (29/09 – 05/10/2026)

Tarea	Fecha	Comentarios
Módulo de aprobación/rechazo de solicitudes (backend + frontend)	02/10/2026	Notificación de estado al estudiante
Módulo de generación de reportes básicos (backend + frontend)	04/10/2026	Estadísticas mínimas solicitadas por el Ministerio
Refinar la consulta del estado de beca para el estudiante	05/10/2026	Ya cubierto parcialmente; se ajusta la vista final

FASE 4 — Pruebas y Calidad (06/10 – 12/10/2026)

Tarea	Fecha	Comentarios
Pruebas unitarias e integración de todos los módulos	09/10/2026	Cobertura mínima acordada en la Definición de Hecho
Anexo 6 — Plan de Aseguramiento de Calidad (QAP)	10/10/2026	
Anexo 7 — Plan de Pruebas	11/10/2026	
Pruebas de regresión general (feature freeze)	12/10/2026	Hito: Objetivo Específico 2 cumplido

FASE 5 — Documentación Final (13/10 – 19/10/2026)

Tarea	Fecha	Comentarios
Manual Técnico	16/10/2026	Instalación, configuración y arquitectura final
Manual de Usuario	18/10/2026	Con capturas de pantalla del sistema funcionando

Actualizar el Anexo 5 con la justificación final de patrones aplicados	19/10/2026	

FASE 6 — Cierre y Entrega (20/10 – 26/10/2026)

Tarea	**Fecha**	**Comentarios**
Revisión general de código y limpieza (principios SOLID)	22/10/2026	
Revisión final del repositorio Git (commits,, historial)	23/10/2026	
Preparar y ensayar la presentación final	24/10/2026	
Hito final: proyecto completo, probado y documentado	26/10/2026	Nueva fecha límite personal

5.4. Fechas clave

Hito	**Fecha**
Inicio del proyecto	08/08/2026
Corte de este documento (cambio a individual)	16/09/2026
Cierre Fase 1 — Módulos base	21/09/2026
Cierre Fase 2 / Objetivo Específico 1	28/09/2026
Cierre Fase 3 — Aprobación y Reportes	05/10/2026
Cierre Fase 4 / Objetivo Específico 2	12/10/2026
Cierre Fase 5 — Documentación final	19/10/2026
Entrega final individual (nueva fecha límite)	26/10/2026
Fecha oficial de entrega al catedrático	31/10/2026

Nuevos requerimientos: octubre 26

Aprobación doble para becas mayores de Q10,000: el evaluador revisa y califica la solicitud; luego, un coordinador la preaprueba y un segundo coordinador distinto registra la aprobación final. Para montos de Q10,000 o menos se mantiene la aprobación de un solo coordinador.

octubre 26

- **Registro de aprobaciones:** se guarda quién tomó cada decisión, cuándo lo hizo y cuál fue el resultado.

octubre 27

- **Auditoría:** se registra qué información relevante cambió, cuál fue el cambio, quién lo realizó y a qué hora. El acceso a la auditoría está restringido al coordinador, y no se deben guardar contraseñas ni tokens en esos registros.

octubre 28

- **Gestión de usuarios:** se agregó un espacio para que el coordinador cree y edite perfiles con los tres roles existentes: estudiante, coordinador y evaluador.
- **Seguimiento y fechas:** se agregó la hora de las solicitudes en las vistas de seguimiento y coordinación.
- **Interfaz de registro:** se mejoró la presentación para que coincida con el diseño y los temas claro y oscuro del proyecto.
- octubre 29
- **Vista de solicitudes:** se ajustó la distribución visual de los estados, mensajes y botones de aprobación para evitar que se encimen.

Anexo 2 Revisión de Sprint y registro de evidencia

Documento preparado para registrar la revisión de un incremento de BecaNet. No constituye una minuta celebrada: no se adjuntaron fechas, asistentes, acuerdos ni aceptación del Product Owner. Se mantiene la estructura del Anexo 2 del curso.

Información general

Dato	Registro
Proyecto	BecaNet
Sprint y fecha de reunión	Pendientes de aportar
Responsable de demostración	Karen Beatriz Canú Cocon
Product Owner académico	Ing. Mario Fuentes; asistencia pendiente de confirmar

1 Objetivo del Sprint

Debe transcribirse el objetivo de la planificación real. Como objetivo propuesto para una revisión integral: demostrar el expediente desde la postulación hasta una decisión con trazabilidad, diferenciando evaluación y aprobación por monto.

2 Revisión de ítems completados

Ítem a revisar	Criterio observable	Estado documental
US007 y US008	Solicitud y certificación adjunta	Resultado de demo no aportado
US011 y US012	Asignación, puntaje y revisión terminada	Resultado de demo no aportado

Ítem a revisar	Criterio observable	Estado documental
US013	Aprobación simple y doble coordinación	Resultado de demo no aportado
US015 a US017	Notificación y reportes contrastables	Resultado de demo no aportado

No se asignan puntos de historia ni se calcula velocidad porque no se aportaron estimaciones ni un Sprint Backlog aprobado.

3 Demostración del incremento

Guion propuesto: usar una convocatoria abierta y cuentas de estudiante, evaluador y dos coordinadores. Crear expedientes independientes de Q10,000 y Q10,000.01, adjuntar certificación, distribuir, evaluar y resolver. Mostrar que el primer expediente requiere una decisión y que el segundo sigue pendiente tras preaprobarse. Intentar que el mismo coordinador complete ambas decisiones y comprobar el rechazo de la operación.

4 Feedback del Product Owner y stakeholders

Terminar con el frontend

5 Análisis y adaptación del backlog

Propuestas derivadas de la revisión documental: verificar autorización en todos los endpoints; alinear el esquema SQL y EF; acreditar los patrones mínimos; documentar ejecución actual de pruebas y completar interfaces pendientes. Son hallazgos de esta revisión, no acuerdos atribuidos al docente.

6 Conclusiones y próximos pasos

Quedan por aportar número de Sprint, fechas, ítems comprometidos, resultados observados, feedback y aceptación. Las próximas reuniones se registrarán cuando estén acordadas. No se emite una declaración de Sprint exitoso sin esos datos.

Anexo 3 Política de trabajo individual

Adaptación de la Política de Trabajo en Grupo al desarrollo individual. Las reglas siguientes son una propuesta de trabajo documentada y no un acuerdo grupal ni una política ya firmada.

1 Información del proyecto y responsable

BecaNet, Análisis de Sistemas II. Responsable: Karen Beatriz Canú Cocon. El docente conserva el rol de Product Owner académico previsto por el enunciado. No se atribuyen integrantes, revisores o firmas adicionales.

2 Misión y objetivos de trabajo

Construir un MVP comprensible, mantenible y trazable que permita demostrar el flujo de becas y las decisiones de diseño. Planificar trabajo según la capacidad individual, separar implementación de verificación y conservar evidencia de los cambios y de sus limitaciones.

3 Ética y responsabilidad

- Usar información y datos de prueba identificados; reconocer fuentes y recursos reutilizados.
- No presentar propuestas, capturas antiguas o mensajes de commits como pruebas actuales de funcionamiento.
- No incorporar contraseñas, tokens o documentación personal real a la entrega pública.
- Comunicar bloqueos o cambios de alcance al docente con evidencia y una propuesta de solución.

4 Participación y seguimiento

La autora asume la carga completa de análisis, implementación, pruebas y documentación. Las tareas deben registrarse con identificador, criterio de aceptación y evidencia. Planning, revisión diaria individual, Review y retrospectiva pueden adaptarse como prácticas de organización; no se afirma que esa adaptación equivalga a Scrum completo ni que las ceremonias históricas hayan ocurrido.

5 Resolución de bloqueos

Registrar el problema, reproducirlo y revisar alternativas técnicas. Si afecta requisitos, aceptación o reglas académicas, elevar una consulta al docente. Las discrepancias documentales se resuelven contrastando plantilla, código y evidencia, conservando la decisión. No corresponden mediación grupal ni sanciones entre compañeros en esta modalidad.

6 Incumplimientos y recuperación

Ante un retraso, registrar impacto, trabajo pendiente y nueva propuesta de entrega. Las consecuencias académicas corresponden al docente; este documento no crea sanciones ni presume una exención de los requisitos de ramas, Pull Requests o revisión entre compañeros.

7 Confidencialidad y aceptación

Mantener datos ficticios para demostración y excluir secretos de capturas, logs y repositorio. La firma y validación de esta adaptación quedan pendientes. Es necesario confirmar con el docente cómo se cumplirá o evaluará la revisión entre pares cuando el trabajo es individual.

Anexo 4 Visión del producto y Product Backlog

1 Información General del Proyecto

Nombre del proyecto: BecaNet

Proyecto asignado: Plataforma Nacional para la Gestión Integral de Becas — Ministerio de Educación

Product Owner académico: Ing. Mario Fuentes, según el enunciado del curso.

Equipo de desarrollo: Karen Beatriz Canú Cocon (proyecto individual; análisis, desarrollo y documentación).

Fecha de consolidación: 30 de octubre de 2026. Se conserva el backlog de 17 historias del documento aportado.

Versión consolidada: 2.0.

2 Visión del Producto

Para estudiantes que buscan apoyo para continuar sus estudios y para el personal del Ministerio de Educación que administra las becas, BecaNet es una plataforma web para gestionar convocatorias, solicitudes, documentación, evaluación y resolución. A diferencia de un trámite disperso entre documentos y gestiones no centralizadas, BecaNet reúne el flujo en un sistema con seguimiento del estado, permisos por rol, registro de decisiones y reportes básicos.

3 Stakeholders y User Personas

Stakeholders identificados

Ministerio de Educación: institución destinataria del sistema y responsable institucional del proceso de becas.

Estudiantes: consultan convocatorias, presentan solicitudes, cargan documentos y siguen el resultado.

Coordinadores: administran convocatorias y perfiles; gestionan asignaciones, revisan auditoría y registran decisiones de aprobación o rechazo.

Evaluadores: revisan solicitudes asignadas, califican y registran observaciones.

Autora y responsable del proyecto: Karen Beatriz Canú Cocon.

Universidad Mariano Gálvez de Guatemala: contexto académico del proyecto.

Personas usuarias

Las siguientes personas son arquetipos funcionales construidos a partir de los roles y pantallas del sistema. No representan personas reales ni incluyen datos demográficos no documentados.

Estudiante postulante

Descripción: Usuario que busca una beca y utiliza las vistas de convocatorias, solicitud, documentación y seguimiento.

Necesidades y metas: Necesita conocer convocatorias abiertas, enviar el motivo y monto solicitado, adjuntar los documentos permitidos y ver los cambios de estado.

Frustraciones o riesgos que el sistema atiende: Necesita que el estado y el resultado de su solicitud sean visibles sin depender de consultas manuales.

Coordinador de becas

Descripción: Usuario encargado de administrar el proceso desde el área de coordinación.

Necesidades y metas: Necesita crear y cerrar convocatorias, gestionar perfiles, organizar comités y asignaciones, registrar decisiones, consultar auditoría y revisar reportes.

Frustraciones o riesgos que el sistema atiende: Debe contar con trazabilidad de quién tomó cada decisión y, para montos superiores a Q10,000, impedir que una sola persona complete las dos aprobaciones.

Evaluador

Descripción: Usuario que califica las solicitudes asignadas por coordinación.

Necesidades y metas: Necesita revisar documentación, registrar puntaje de 0 a 100 y observaciones, y marcar la revisión como completada.

Frustraciones o riesgos que el sistema atiende: La evaluación debe quedar separada de la aprobación administrativa; el evaluador no realiza las aprobaciones de coordinación.

4 Product Backlog Inicial.

Épicas

EPIC-01 — Acceso y gestión de perfiles

EPIC-02 — Administración de convocatorias

EPIC-03 — Solicitudes y documentación

EPIC-04 — Comités y asignación de evaluaciones

EPIC-05 — Evaluación y resolución de solicitudes

EPIC-06 — Seguimiento, auditoría y reportes

US-001 EPIC-01

Como estudiante, quiero crear una cuenta con mis datos de perfil para poder postularme a una beca.

Criterios de aceptación: El registro público crea cuentas de estudiante; se solicitan los datos definidos por el formulario y se rechaza un correo que ya exista.

US-002 EPIC-01

Como usuario registrado, quiero iniciar sesión para acceder a las funciones correspondientes a mi rol.

Criterios de aceptación: El inicio de sesión admite los roles Estudiante, Coordinador y Evaluador; una autenticación válida entrega la sesión/token usado por el cliente.

US-003 EPIC-01

Como usuario, quiero actualizar los datos editables de mi perfil para mantenerlos vigentes.

Criterios de aceptación: Se pueden actualizar los campos de perfil disponibles para el tipo de usuario; una contraseña nueva se almacena mediante el hasher y nunca se devuelve en la respuesta.

US-004 EPIC-02

Como coordinador, quiero crear una convocatoria con sus datos, requisitos y fechas para abrir una oportunidad de beca.

Criterios de aceptación: La convocatoria queda asociada al coordinador y comienza en estado Borrador.

US-005 EPIC-02

Como coordinador, quiero publicar o cerrar una convocatoria para controlar cuándo se reciben solicitudes.

Criterios de aceptación: La publicación cambia la convocatoria a Abierta; el cierre la cambia a Cerrada y el sistema deja de admitir nuevas postulaciones a esa convocatoria.

US-006 EPIC-02

Como estudiante, quiero consultar las convocatorias abiertas para identificar aquellas a las que puedo postularme.

Criterios de aceptación: La consulta de convocatorias para estudiantes devuelve las que se encuentran en estado Abierta.

US-007 EPIC-03

Como estudiante, quiero enviar una solicitud asociada a una convocatoria para participar en el proceso de beca.

Criterios de aceptación: La convocatoria debe estar Abierta; se registra motivo y monto solicitado; no se admite más de una solicitud activa del mismo estudiante para la misma convocatoria.

US-008 EPIC-03

Como estudiante, quiero adjuntar la documentación requerida a mi solicitud para respaldar mi postulación.

Criterios de aceptación: Se admiten archivos PDF, JPG o PNG de hasta 5 MB; se puede identificar la certificación de estudios como tipo documental.

US-009 EPIC-03

Como estudiante, quiero cancelar mi solicitud mientras la convocatoria siga abierta para retirar mi postulación.

Criterios de aceptación: Solo se permite cancelar una solicitud propia mientras su convocatoria esté Abierta; el cambio de estado se registra y se notifica.

US-010 EPIC-04

Como coordinador, quiero conformar un comité evaluador para organizar la revisión de solicitudes.

Criterios de aceptación: El comité se configura con evaluadores registrados y requiere al menos dos integrantes.

US-011 EPIC-04

Como coordinador, quiero asignar solicitudes a un comité y distribuirlas entre evaluadores para organizar la carga de trabajo.

Criterios de aceptación: La asignación se realiza desde coordinación; existe distribución automática y asignación a evaluador; las solicitudes sin la certificación de estudios requerida se omiten de la distribución.

US-012 EPIC-05

Como evaluador, quiero revisar una solicitud asignada, registrar puntaje y observaciones, y marcarla como revisada para entregar mi evaluación.

Criterios de aceptación: El puntaje está entre 0 y 100; la evaluación se registra para una solicitud asignada; al completarse, el estado de revisión queda como Revisado. Esta historia no concede al evaluador la aprobación de coordinación.

US-013 EPIC-05

Como coordinador, quiero aprobar o rechazar una solicitud ya evaluada para registrar la resolución de la beca.

Criterios de aceptación: Hasta Q10,000 inclusive, decide un coordinador; por encima de Q10,000, un coordinador registra la preaprobación y otro coordinador distinto registra la aprobación final. La solicitud no queda aprobada antes de ambas decisiones; un rechazo detiene el flujo. Se registra actor, rol, fecha/hora UTC y resultado.

US-014 EPIC-01

Como coordinador, quiero crear, consultar y editar perfiles de estudiante, coordinador y evaluador para mantener las cuentas necesarias para el proceso.

Criterios de aceptación: La gestión de usuarios está restringida al rol Coordinador; se validan los correos duplicados y una modificación de perfil se registra en auditoría sin incluir secretos.

US-015 EPIC-06

Como estudiante, quiero recibir y consultar notificaciones de cambios en mis solicitudes para conocer su avance.

Criterios de aceptación: Las notificaciones se asocian al estudiante y a la solicitud e indican el estado anterior, el nuevo y la fecha/hora del cambio.

US-016 EPIC-06

Como coordinador, quiero consultar un reporte de resultados por convocatoria para revisar el resultado del proceso.

Criterios de aceptación: El reporte presenta conteos de solicitudes y sus resultados para una convocatoria específica.

US-017 EPIC-06

Como coordinador, quiero consultar estadísticas generales para obtener un resumen global de las solicitudes.

Criterios de aceptación: El sistema presenta estadísticas agregadas del conjunto de solicitudes, sin sustituir el detalle de cada expediente.

Trazabilidad de auditoría transversal

Las operaciones relevantes registran actor, rol, operación, fecha/hora y resultado o cambio, según el evento. Los endpoints de consulta de auditoría están restringidos al rol Coordinador. Las contraseñas y tokens no deben almacenarse en esos registros; los detalles sensibles se omiten o se redactan por seguridad.

5 Requisitos No Funcionales

Seguridad y control de acceso: JWT y autorización por rol donde se declaran; no se acredita cobertura de autorización completa de la API. Las contraseñas se procesan con PBKDF2 y SHA-256 con sal aleatoria.

Protección de secretos y trazabilidad: Los registros de auditoría documentan actor, operación, cambio o resultado y fecha/hora, evitando almacenar contraseñas, tokens u otros secretos.

Restricción de archivos: La carga documental permite PDF, JPG y PNG, con tamaño máximo de 5 MB por archivo.

Usabilidad: La interfaz separa las funciones por rol y permite al estudiante consultar estados y notificaciones. El frontend incluye modo claro y oscuro.

Rendimiento, disponibilidad y compatibilidad: En el repositorio y Anexo 1 consultados no se especifican objetivos medibles de rendimiento, disponibilidad ni una matriz de navegadores. Estos parámetros quedan pendientes de definición antes de una puesta en producción.

6 Definition of Done

Criterios propuestos para considerar completa una historia de usuario del proyecto. Son una lista de verificación y no una afirmación de que cada punto haya sido validado para todas las historias.

☐ La funcionalidad corresponde a la historia y satisface todos sus criterios de aceptación.

☐ Los accesos se verifican por rol y la información sensible no se expone en respuestas ni en registros de auditoría.

☐ Los cambios de datos y decisiones relevantes conservan la trazabilidad definida para el sistema.

☐ Se ejecutan y documentan las pruebas aplicables; los resultados se relacionan con los criterios de aceptación.

☐ El cambio se integra con Angular, ASP.NET Core y SQL Server sin alterar los módulos existentes fuera de su alcance.

☐ La documentación de usuario o técnica afectada se actualiza.

☐ La autora realiza una revisión final del cambio y registra pendientes conocidos antes de presentarlo.

7 Priorización y trazabilidad de la revisión

Prioridades propuestas para el cierre, pendientes de validación del Product Owner: primero reglas de autorización, integridad de datos y decisiones por monto; después recorridos completos y reportes; finalmente mejoras visuales. No se asignan retrospectivamente puntos ni se declara aceptada ninguna historia por la sola presencia de código.

La US013 cubre la regla de monto y la US014 la administración de perfiles. La consulta de auditoría se formaliza como requisito transversal RT01 para hacer visible su verificación, sin alterar artificialmente el total histórico de 17 historias.

Anexo 5 Arquitectura y diseño del sistema

1 Información general

Diseño de BecaNet revisado por Karen Beatriz Canú Cocon en modalidad individual. La versión original del Anexo 5 es del 4 de septiembre de 2026; esta consolidación corrige referencias a cinco integrantes, API Gateway y cantidades fijas de tablas que no describen de forma uniforme el código actual.

2 Propuesta de arquitectura

2 1 Estilo seleccionado

Arquitectura por capas lógicas. Angular se ejecuta en el navegador como cliente independiente. El backend reúne controladores, servicios y acceso a datos en un proyecto ASP.NET Core; SQL Server y el almacenamiento de archivos son recursos externos al navegador. No se identificó un API Gateway dedicado ni microservicios desplegables de manera independiente.

2 2 Justificación y compromisos

La separación facilita localizar reglas de becas, cambiar las pantallas sin reescribir la persistencia y probar servicios con dobles de prueba. Para una única autora reduce la complejidad de despliegue frente a microservicios. El costo es mantener DTO y contratos y evitar que las reglas se dispersen entre componentes.

El uso de repositorios no significa aislamiento absoluto: varios servicios también dependen directamente de BecaNetDbContext. Esto simplifica consultas y transacciones locales, pero aumenta el acoplamiento a EF Core. SOLID se emplea como criterio de revisión, no como certificación de que cada clase cumple todos sus principios.

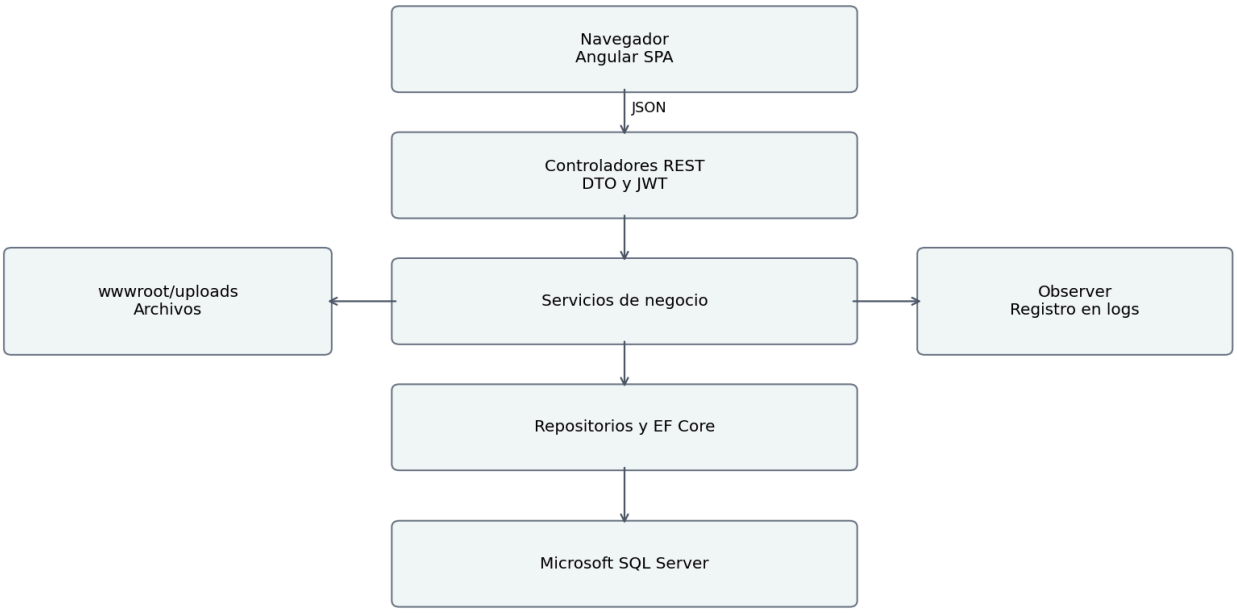


Figura 1. Vista lógica actualizada. Los servicios pueden acceder también al DbContext. JWT interviene en las operaciones protegidas; el diagrama no implica autorización uniforme de toda la API.

3 Diagramas esenciales y casos de uso

3 1 Vista de casos de uso

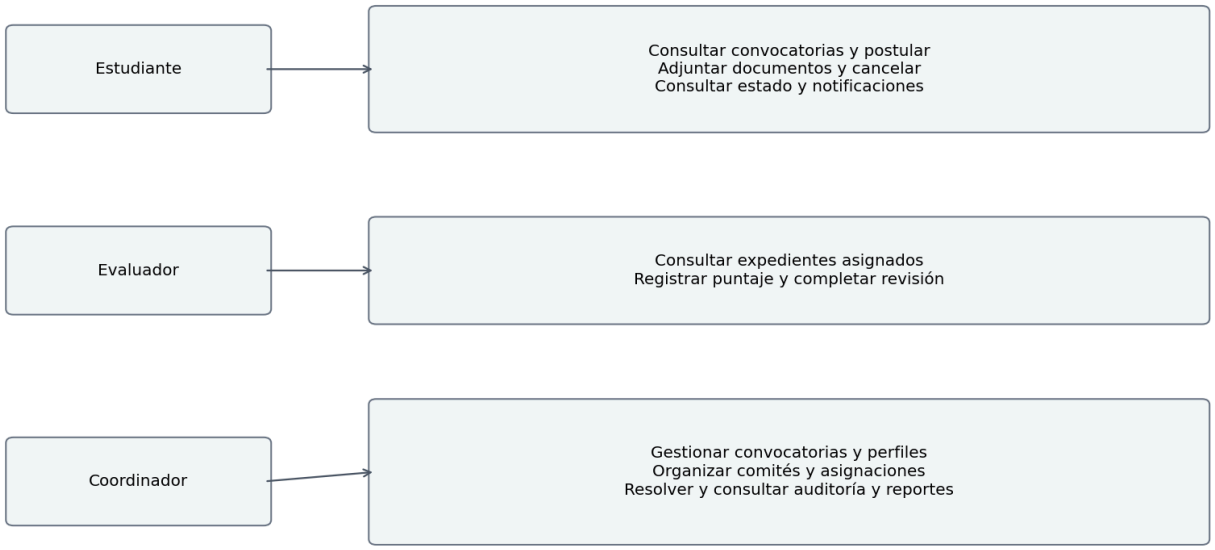




Figura 2. Vista funcional por actor. El detalle siguiente especifica precondiciones y resultados; no se atribuye aprobación al evaluador.

CU01 Acceder al sistema

Actor: Usuario registrado. Precondición: Credenciales válidas y API disponible.

Flujo principal: Introducir credenciales; validar identidad; obtener token y mostrar funciones de rol.

Alternativa o excepción: Si las credenciales son inválidas, denegar acceso sin revelar secretos.

Postcondición: Sesión lógica iniciada; no concede permisos de otros roles.

CU02 Administrar convocatoria

Actor: Coordinador. Precondición: Cuenta válida y permisos de coordinación.

Flujo principal: Crear convocatoria; registrar requisitos y fechas; publicar; cerrar cuando corresponda.

Alternativa o excepción: No admitir postulación si el estado no es ABIERTA.

Postcondición: Convocatoria en el estado solicitado. La protección de los endpoints debe completarse.

CU03 Presentar solicitud

Actor: Estudiante. Precondición: Sesión y convocatoria abierta.

Flujo principal: Elegir convocatoria; registrar motivo y monto; enviar solicitud propia.

Alternativa o excepción: Rechazar duplicidad activa; revisar conflicto del índice tras cancelación.

Postcondición: Solicitud EN_PROCESO vinculada al estudiante y convocatoria.

CU04 Adjuntar documentación

Actor: Estudiante. Precondición: Solicitud propia y archivo permitido.

Flujo principal: Elegir documento; marcar tipo; cargar; verificar registro y acceso al archivo.

Alternativa o excepción: Rechazar archivo vacío, extensión no permitida o tamaño excesivo.

Postcondición: Archivo almacenado, metadatos y evento de auditoría.

CU05 Organizar revisión

Actor: Coordinador. Precondición: Evaluadores registrados y expedientes elegibles.

Flujo principal: Crear comité con dos o más evaluadores; asociar solicitudes y distribuir carga.

Alternativa o excepción: Excluir expedientes sin certificación; registrar los no asignados.

Postcondición: Comité y asignación disponibles para revisión.

CU06 Evaluar expediente

Actor: Evaluador. Precondición: Solicitud asignada al usuario.

Flujo principal: Consultar expediente; registrar puntaje de 0 a 100 y observaciones; completar revisión.

Alternativa o excepción: Rechazar puntaje fuera de rango y acceso a expediente ajeno.

Postcondición: Expediente evaluado y revisión completada según el servicio.

CU07 Resolver expediente

Actor: Coordinador. Precondición: Estado EVALUADA; para monto mayor, revisión REVISADO.

Flujo principal: Hasta Q10,000 resolver; por encima, preaprobar y usar otro coordinador para decisión final.

Alternativa o excepción: Rechazo exige motivo; impedir segunda decisión por la misma identidad.

Postcondición: APROBADA o RECHAZADA; preaprobación conserva estado pendiente.

CU08 Seguir y cancelar

Actor: Estudiante. Precondición: Solicitud propia; convocatoria abierta para cancelar.

Flujo principal: Consultar estado y notificaciones; si corresponde, confirmar cancelación.

Alternativa o excepción: Denegar cancelación de expediente ajeno o convocatoria cerrada.

Postcondición: Estado visible o CANCELADA con trazabilidad.

CU09 Consultar auditoría

Actor: Coordinador. Precondición: Token con rol COORDINADOR.

Flujo principal: Consultar operaciones generales o por solicitud.

Alternativa o excepción: Sin token o rol autorizado, denegar.

Postcondición: Eventos consultables sin contraseñas ni tokens.

CU10 Consultar reportes

Actor: Coordinador como actor previsto. Precondición: Datos de solicitudes y convocatoria existente.

Flujo principal: Solicitar conteos de una convocatoria o estadísticas generales.

Alternativa o excepción: Informar convocatoria inexistente; comprobar protección API.

Postcondición: Resumen por estados; no se presume exportación ni pantalla específica.

3 2 Componentes y contratos

Componente	Responsabilidad	Referencia de código
Angular	Navegación, formularios y HTTP	src/app/app.routes.ts; services/
Controladores	Endpoints y validación de identidad declarada	BecaNet.Api/Controllers/
Servicios	Reglas del proceso de becas	BecaNet.Api/Services/
Repositorios	Consultas y persistencia de entidades	BecaNet.Api/Repositories/
DbContext	Mapeo, relaciones e índices	Data/BecaNetDbContext.cs
Seguridad y eventos	JWT, hash y observación de cambios	Security/ y Observers/

3 3 Modelo de dominio

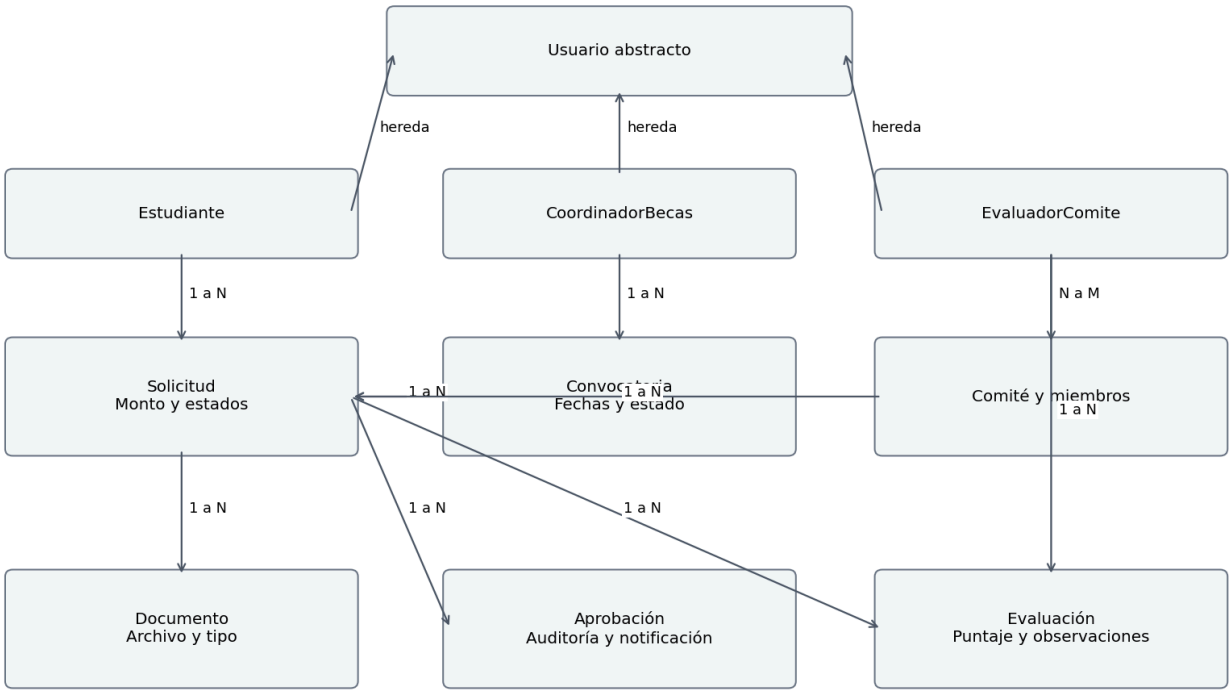


Figura 3. Modelo conceptual resumido del código. Los registros de auditoría pueden referirse a otras entidades y no solo a solicitudes. No es un inventario físico de tablas.

3 4 Datos y diferencias de esquema

El modelo de ejecución mapea los tres tipos de usuario y las entidades de convocatoria, solicitud, documento, comité, membresía, evaluación, aprobación, auditoría y notificación. Las clases de estados son constantes string, no enumeraciones C#. No existe una entidad Reporte en Models; los reportes actuales se calculan con consultas y DTO.

Elemento	Datos relevantes	Relación o restricción
Usuario y perfiles	Id, nombre, correo, hash, teléfono; campos de rol	Herencia en C# y mapeos propios por tipo
Convocatoria	Título, requisitos, fechas, estado, coordinador	Un coordinador puede tener varias convocatorias
Solicitud	Estudiante, convocatoria, monto, motivo, estado, evaluador	Índice único estudiante y convocatoria; monto decimal 12,2
Documento	Solicitud, tipo, nombre, ruta y fecha	Dependencia de solicitud; eliminación en cascada configurada
Comité y membresía	Comité y evaluadores	Clave compuesta en ComiteMiembro

Elemento	Datos relevantes	Relación o restricción
Evaluación	Solicitud, evaluador, puntaje y observaciones	Índice único por solicitud y evaluador
AprobacionSolicitud	Tipo, actor, resultado, motivo y fecha UTC	Un registro por solicitud y tipo de decisión
RegistroAuditoria	Entidad, actor, operación, detalle y fecha UTC	Solicitud opcional; índices de consulta
NotificacionSolicitud	Estudiante, solicitud, estados y fecha UTC	Evento persistido de cambio de estado

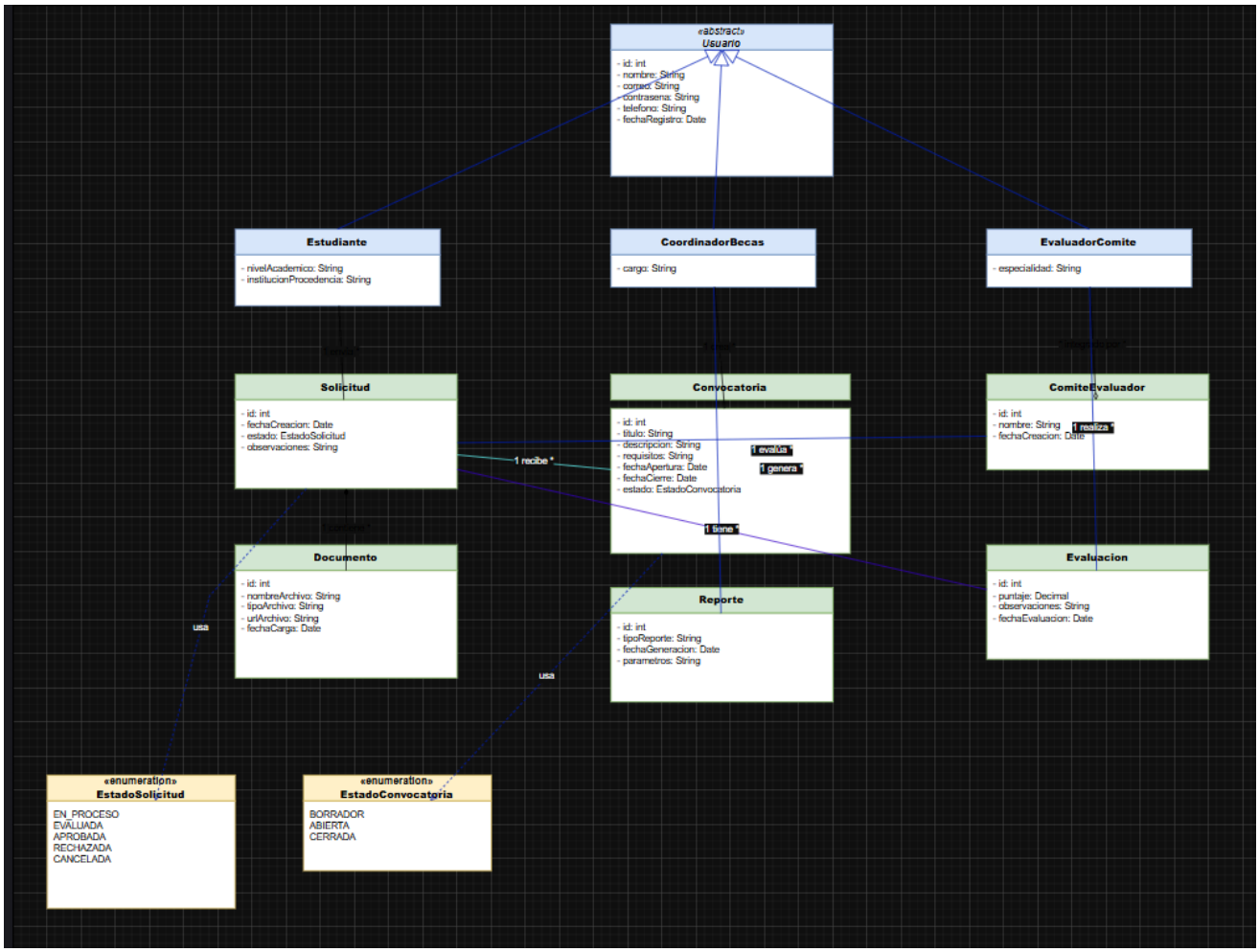
El script inicial contiene 11 CREATE TABLE, con Usuario y perfiles físicos prefijados tbl_, vistas unificadas y una tabla Reporte. El script BecaNet_Nuevos_Requisitos.sql incorpora otras tres tablas. El DbContext usa ToTable para los perfiles, pero no explicita una jerarquía TPT completa de la misma forma que la describe el documento antiguo. Por ello, no se declara un único total de tablas ni compatibilidad automática entre ambos caminos de creación.

EnsureCreated no actualiza esquemas existentes; Program.cs ejecuta después el parche SQL. Debe comprobarse en una base desechable que tablas, vistas, nombres, claves e inserciones de perfiles coinciden con el modelo de EF. No se prescribe dotnet ef database update sin confirmar que existen migraciones utilizables.

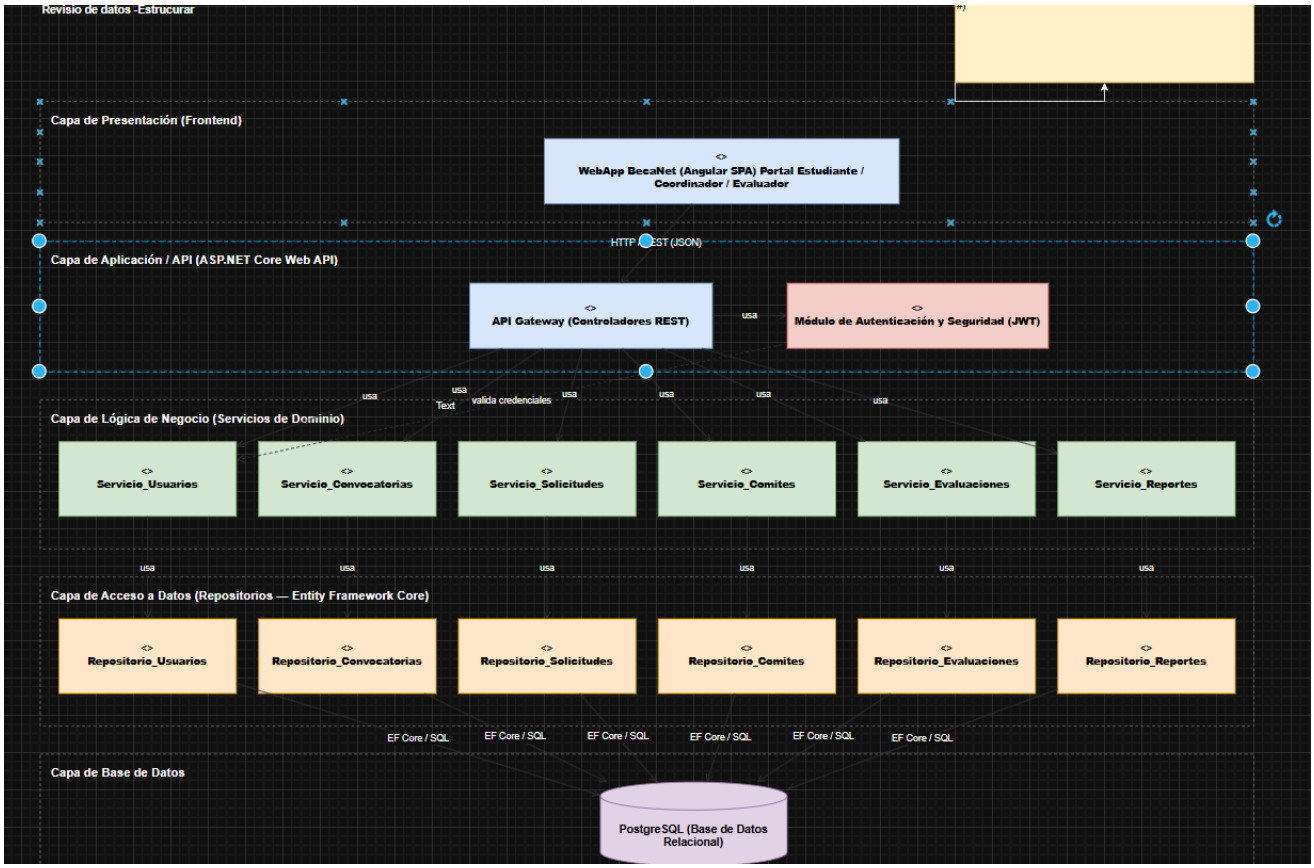
4 Evolución del diseño

Los draw.io del repositorio se conservan como antecedentes editables, pero las tres figuras de esta versión se redibujaron a partir del código revisado. La separación evaluación y decisión, el umbral de monto y los registros persistentes deben mantenerse sincronizados en diagramas, backlog, pruebas y manuales. Cualquier ampliación requiere analizar el impacto y conservar su fuente.

Diagramas utilizados

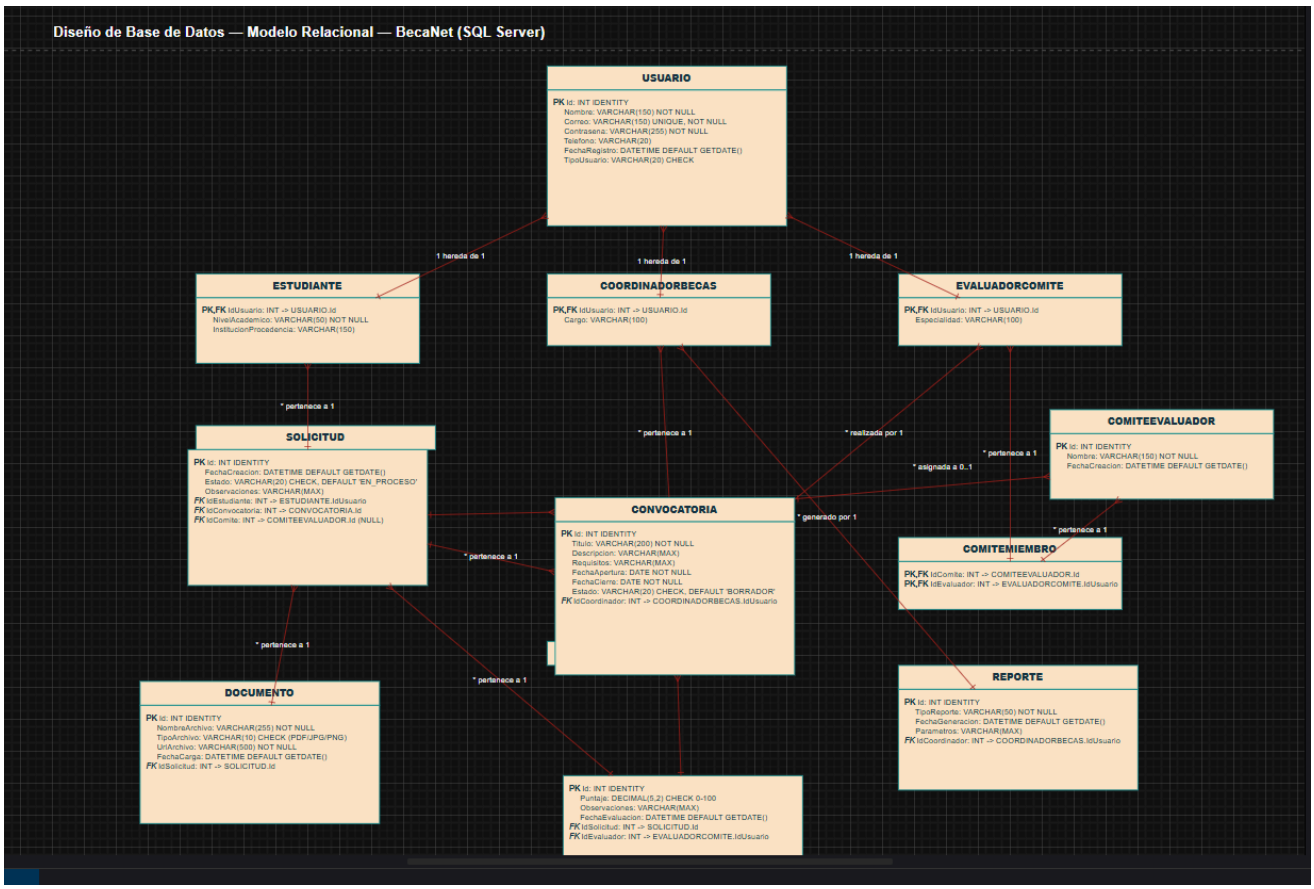
- Diagrama de clases
- 

- Diagram de componentes



-

- Modelo relacional



-

Anexo 6 Plan de aseguramiento de la calidad

Información general

Responsable de calidad: Karen Beatriz Canú Cocon. Product Owner académico: Ing. Mario Fuentes.

1 Propósito y alcance

Definir revisiones y evidencias para que requisitos, código, datos y documentación coincidan. El plan cubre frontend, API, SQL, archivos, pruebas y entrega académica. Las medidas que siguen constituyen criterios de trabajo y cierre; su cumplimiento se acreditará mediante registros, no por declaración general.

2 Productos sujetos a calidad

Producto	Criterio de revisión	Evidencia esperada
Backlog y casos de uso	Reglas coherentes, comprobables y por rol	Historia y caso de prueba enlazados
Diseño y diagramas	Corresponden al commit entregado	Archivos y decisiones de diseño
Código y SQL	Compilan y respetan integridad y permisos	Build y prueba en SQL Server
Pruebas	Cubren reglas y regresión	Salida fechada con commit
Manuales	Pasos reproducibles y límites visibles	Recorrido documentado
Incremento	Cumple DoD y criterios de aceptación	Registro de demo y validación

3 Normas y prácticas aplicables

Aplicar nombres consistentes en C# y TypeScript y conservar .editorconfig como regla básica de formato. No confundir ese archivo con un linter. Revisar que los DTO no devuelvan hashes, que las rutas no dependan solo de controles visuales y que cada transición crítica registre actor y resultado. El DoD del Anexo 4 es la referencia común.

4 Acciones y tareas de calidad

- Antes de desarrollar: contrastar historia, regla de negocio, permisos y datos requeridos.
- Durante el cambio: revisar responsabilidades de clase, validaciones, manejo de errores y efectos sobre el esquema.
- Antes de integrar: ejecutar pruebas pertinentes, revisar el diff y registrar pendientes. La autorrevisión no se presenta como revisión independiente.
- Antes de entregar: repetir recorrido por rol, validar límites de monto y archivos y actualizar los manuales.
- Al cerrar una fase: registrar qué falló, causa, acción correctiva y resultado posterior; no reconstruir retrospectivas ficticias.

5 Herramientas y métodos

Herramienta	Situación observada o documentada
Git y GitHub	Repositorio revisado en main; clonación superficial usada solo para inspección. No se verificó cumplimiento histórico de ramas o PR.
ClickUp	Espacio Análisis - Becas citado en documentos; tablero no consultado.
Pruebas backend	Proyecto net9.0 con NUnit y xUnit, Moq, InMemory y coverlet.collector.
Pruebas frontend	Karma/Jasmine y archivos spec.ts.
Swagger	Configurado solo en entorno Development. Postman puede utilizarse, sin colección verificada.
CI y análisis estático	No se encontró workflow CI ni ESLint en la revisión. Incorporarlos es una mejora propuesta.

6 Administración de configuración

Identificar cada entrega por commit. Como práctica propuesta, desarrollar en ramas de tarea y usar Pull Requests antes de main. No se declara que develop o GitFlow se hayan usado. El enunciado prohíbe trabajo directo en main y exige revisión entre compañeros; la adaptación de esa exigencia para trabajo individual requiere confirmación del docente.

El entorno confirmado por archivos es de desarrollo local. Staging y producción no están acreditados. Separar cadenas de conexión, claves JWT y datos de prueba; las cuentas predeterminadas y la creación de datos al arrancar requieren revisión antes de publicar. Respaldar SQL Server y uploads de manera coordinada; la existencia de código en Git no respalda los expedientes de una base en uso.

7 Registros y métricas

Cada defecto debe contener ID, historia, commit, entorno, pasos, resultado esperado, resultado obtenido, severidad, responsable y evidencia del cierre. Mantener los resultados de pruebas con fecha, comando, total ejecutado, aprobadas, fallidas y omitidas. No asignar cobertura mínima o rendimiento alcanzado sin medición.

8 Roles y responsabilidades

La autora analiza, implementa, verifica y mantiene evidencias; el docente valida el alcance y la aceptación académica.

Criterios propuestos de liberación académica

El recorrido crítico debe funcionar con datos de prueba y permisos comprobados; las pruebas aplicables deben tener resultados registrados; no deben quedar defectos que permitan decisiones no autorizadas o alteren el flujo de doble coordinación. Los pendientes restantes deben quedar visibles y el docente debe conocer cualquier desviación del enunciado. No se emite aquí un acta de liberación.

Anexo 7 Plan de pruebas

Información general y estado

Plan de pruebas de BecaNet, responsable Karen Beatriz Canú Cocon. Los casos detallados de este anexo fueron definidos para la consolidación.

1 Propósito

Verificar criterios de aceptación y detectar errores en el recorrido completo, con énfasis en identidad, expedientes, carga documental, evaluación y doble coordinación. Cada resultado debe relacionarse con la versión del código y los datos utilizados.

2 Objetivos

- Comprobar el alcance de las 17 historias y el requisito transversal de auditoría.
- Probar casos positivos, negativos y valores límite sin usar expedientes personales reales.
- Separar pruebas unitarias de integración real con SQL Server y de aceptación en navegador.
- Conservar resultados reproducibles y repetir regresión cuando se corrijan defectos.

3 Alcance y exclusiones

Se incluyen registro, login, perfiles, convocatorias, solicitudes, documentos, comités, asignaciones, evaluación, decisiones, notificaciones, auditoría, reportes y tema visual. No se acredita pentesting profesional, carga masiva, disponibilidad productiva, soporte de navegadores obsoletos ni pruebas de infraestructura nacional. Estas exclusiones no sustituyen las comprobaciones básicas de autorización.

4 Entorno y configuración

Preparar entorno local con API net8.0, herramientas capaces de compilar el proyecto de pruebas net9.0, Angular del lockfile y una instancia SQL Server de prueba. Registrar las versiones realmente usadas; no basta indicar versiones últimas. Los navegadores deben anotarse por nombre y versión al ejecutar. No hay staging confirmado.

Datos mínimos propuestos: un estudiante, dos coordinadores con correos e identificadores distintos, al menos dos evaluadores, una convocatoria abierta, otra cerrada y expedientes separados para cada prueba de decisión. Preparar archivos permitidos, prohibidos, vacíos, de 5,242,880 bytes y de 5,242,881 bytes. El arranque crea una sola cuenta coordinadora predeterminada: la segunda debe prepararse expresamente.

5 Estrategia de pruebas

Unitarias: aislar servicios y reglas con Moq e InMemory donde corresponda. Integración: usar SQL Server real para índices, vistas, relaciones y transacciones; InMemory no demuestra compatibilidad con SQL Server. Sistema: ejecutar la SPA y la API juntas y contrastar la vista con los registros persistidos. Seguridad: probar la API directamente, incluso sin pasar por Angular.

Criterios de entrada: commit identificado, dependencias restauradas, datos de prueba y servicios disponibles. Criterios de salida propuestos: reglas críticas verificadas, fallos corregidos o reconocidos, resultados conservados y aceptación académica pendiente de la persona competente. La existencia de tests en Git no cumple por sí sola el criterio de salida.

Casos funcionales y de límites

CP01 US001 y US002

Acción: Registrar estudiante e iniciar sesión; repetir correo.

Resultado esperado: Cuenta válida inicia sesión; duplicado se rechaza.

Estado: No ejecutado. Evidencia y resultado obtenido: pendientes.

CP02 US002 y US014

Acción: Invocar usuarios con estudiante, evaluador y sin token.

Resultado esperado: Solo coordinador accede; los demás se rechazan.

Estado: No ejecutado. Evidencia y resultado obtenido: pendientes.

CP03 US004 a US007

Acción: Crear, publicar y cerrar convocatoria; intentar postular en cada estado.

Resultado esperado: Solo ABIERTA permite postulación.

Estado: No ejecutado. Evidencia y resultado obtenido: pendientes.

CP04 US007

Acción: Enviar dos solicitudes del mismo estudiante a la misma convocatoria.

Resultado esperado: Se rechaza la segunda activa sin duplicar datos.

Estado: No ejecutado. Evidencia y resultado obtenido: pendientes.

CP05 US007 y US009

Acción: Cancelar y volver a postular en la misma convocatoria.

Resultado esperado: Registrar discrepancia servicio e índice; no declarar aprobado sin definir regla.

Estado: No ejecutado. Evidencia y resultado obtenido: pendientes.

CP06 US008

Acción: Subir PDF, JPG, JPEG y PNG válidos.

Resultado esperado: Archivo, metadatos y tipo documental quedan asociados.

Estado: No ejecutado. Evidencia y resultado obtenido: pendientes.

CP07 US008

Acción: Subir archivo vacío y extensión no permitida.

Resultado esperado: Se rechazan sin crear un registro válido.

Estado: No ejecutado. Evidencia y resultado obtenido: pendientes.

CP08 US008

Acción: Subir archivo de 5,242,880 bytes y otro de 5,242,881.

Resultado esperado: El límite se admite; el byte adicional se rechaza.

Estado: No ejecutado. Evidencia y resultado obtenido: pendientes.

CP09 US009

Acción: Cancelar solicitud propia abierta y ajena o cerrada.

Resultado esperado: Solo cancelación permitida; registro de cambio verificable.

Estado: No ejecutado. Evidencia y resultado obtenido: pendientes.

CP10 US010

Acción: Crear comité con uno y con dos evaluadores.

Resultado esperado: Uno se rechaza; dos integrantes válidos se admiten.

Estado: No ejecutado. Evidencia y resultado obtenido: pendientes.

CP11 US011

Acción: Distribuir expedientes con y sin certificación.

Resultado esperado: Se asignan elegibles; se omiten los incompletos.

Estado: No ejecutado. Evidencia y resultado obtenido: pendientes.

CP12 US012

Acción: Evaluador intenta revisar expediente ajeno.

Resultado esperado: La API rechaza acceso u operación no autorizada.

Estado: No ejecutado. Evidencia y resultado obtenido: pendientes.

CP13 US012

Acción: Registrar puntajes -1, 0, 100 y 101 en expedientes separados.

Resultado esperado: Solo valores de 0 a 100 se admiten.

Estado: No ejecutado. Evidencia y resultado obtenido: pendientes.

CP14 US013

Acción: Resolver solicitud EVALUADA por Q10,000.

Resultado esperado: Una decisión coordinadora produce resolución final.

Estado: No ejecutado. Evidencia y resultado obtenido: pendientes.

CP15 US013

Acción: Preaprobar solicitud por Q10,000.01 con revisión completada.

Resultado esperado: Se registra preaprobación; aún no APROBADA.

Estado: No ejecutado. Evidencia y resultado obtenido: pendientes.

CP16 US013

Acción: Primer coordinador intenta la decisión final del mismo expediente.

Resultado esperado: Operación denegada, sin aprobación final ni cambio indebido.

Estado: No ejecutado. Evidencia y resultado obtenido: pendientes.

CP17 US013

Acción: Segundo coordinador distinto aprueba tras CP15.

Resultado esperado: APROBADA con dos decisiones y trazabilidad.

Estado: No ejecutado. Evidencia y resultado obtenido: pendientes.

CP18 US013

Acción: Rechazar sin motivo y luego con motivo en expediente independiente.

Resultado esperado: Sin motivo se rechaza la operación; con motivo queda RECHAZADA.

Estado: No ejecutado. Evidencia y resultado obtenido: pendientes.

CP19 US013

Acción: Intentar aprobar expediente ya rechazado o sin evaluar.

Resultado esperado: No se permite avanzar hacia aprobación.

Estado: No ejecutado. Evidencia y resultado obtenido: pendientes.

CP20 US015 y RT01

Acción: Cambiar estado y consultar notificaciones y auditoría.

Resultado esperado: Evento del estudiante y trazabilidad con actor y fecha, sin secretos.

Estado: No ejecutado. Evidencia y resultado obtenido: pendientes.

CP21 US016 y US017

Acción: Crear datos de conteo conocido y pedir ambos reportes.

Resultado esperado: Conteos por estado y convocatoria coinciden con los datos.

Estado: No ejecutado. Evidencia y resultado obtenido: pendientes.

CP22 Transversal

Acción: Acceder directo a endpoints de convocatorias, comités, perfil y reportes sin token.

Resultado esperado: Comparar con autorización requerida y registrar accesos permitidos como defectos.

Estado: No ejecutado. Evidencia y resultado obtenido: pendientes.

CP23 US008

Acción: Abrir URL /uploads de un archivo sin sesión.

Resultado esperado: Evaluar exposición; los archivos privados deben requerir autorización.

Estado: No ejecutado. Evidencia y resultado obtenido: pendientes.

CP24 Interfaz

Acción: Cambiar tema, navegar por rol y recargar.

Resultado esperado: Texto legible y rutas previstas; registrar navegador y observación real.

Estado: No ejecutado. Evidencia y resultado obtenido: pendientes.

6 Entregables de prueba

Conservar salida de dotnet test y npm test, registro manual de CP01 a CP24, defectos y retest, capturas pertinentes sin datos sensibles y aceptación del Product Owner si se produce. El informe debe distinguir pruebas descubiertas, ejecutadas, omitidas y fallidas, además de la cobertura medida si se obtiene. No se reporta un porcentaje supuesto.

Anexo 8 Patrones de diseño y control de objetivos

1 Criterio académico y clasificación

El enunciado exige al menos dos patrones creacionales, dos estructurales y dos de comportamiento, cada uno con problema, alternativas, selección y ventajas. No basta nombrar seis técnicas. Repository es un patrón de acceso a datos y no forma parte del catálogo estructural GoF; DTO o un método de mapeo no constituyen por sí solos Adapter. Registrar un servicio con AddSingleton acredita un tiempo de vida compartido en el contenedor, pero no toda la implementación clásica de Singleton.

Patrón o técnica	Evidencia actual	Conclusión de revisión
Repository	Interfaces y clases en Repositories/	Implementado como separación de persistencia; clasificación estructural académica por validar.
Observer	ISolicitudObserver y SolicitudNotificador	Implementado; observadores concretos escriben en ILogger.
Ciclo Singleton	AddSingleton en Program.cs	Instancia por contenedor; justificar si el docente lo acepta como patrón creacional.
Factory Method	SolicitudService crea new Solicitud directamente	No se acreditó Factory Method.
Adapter	MapearADto dentro de servicios	No se acreditó un adaptador independiente.
Strategy	Validación de extensión y tamaño en DocumentoService	No se acreditó familia de estrategias intercambiables.

2 Justificación de las técnicas observadas

Repository

Problema: separar consultas persistentes de la lógica del proceso. Alternativa: utilizar DbContext en todos los servicios o consulta directa en los controladores. Selección: interfaces de repositorio permiten sustituir dependencias en pruebas y centralizar operaciones. Ventaja: menor repetición de consultas. Límite: el uso directo de DbContext en algunos servicios conserva acoplamiento; Repository no debe contarse automáticamente como patrón GoF estructural.

Observer

Problema: reaccionar a cambios de estado sin incrustar todos los canales dentro de cada decisión. Alternativa: invocar cada acción directamente desde el servicio. Selección: notificador con suscriptores ISolicitudObserver. Ventaja: admite observadores adicionales. Límite: los actuales simulan notificación y auditoría mediante logs; la persistencia de registros se realiza también desde los servicios. No hay evidencia de envío de correo real.

Instancia compartida mediante inyección de dependencias

Problema: mantener un notificador y suscripciones comunes durante la ejecución. Alternativa: crear instancias por petición y reconstruir suscriptores. Selección: AddSingleton para el notificador y observadores. Ventaja: comparte la configuración en el contenedor. Límite: no es una demostración suficiente de dos patrones creacionales ni sustituye el análisis de concurrencia y dependencias.

3 Propuestas pendientes de implementación

Las propuestas siguientes documentan alternativas posibles. No se presentan como código existente ni se modificó el repositorio en esta tarea.

Candidato	Problema y alternativa	Justificación y condición
Factory Method creacional	Variar creación según un tipo de solicitud; alternativa new directo.	Usar creadores con una operación de creación sustituible solo si hay variaciones reales. Una clase factory estática sería fábrica simple, no necesariamente Factory Method.
Builder creacional	Construir un resultado complejo con variantes; alternativa constructor o inicializador.	Puede ordenar construcción de reportes con secciones opcionales. Necesita uso real y pruebas para justificarlo.
Adapter estructural	Convertir una interfaz incompatible; alternativa acoplar clientes a ella.	Exige una interfaz objetivo y un adaptado identificables. Extraer MapearADto sin ese problema no basta.
Decorator estructural	Añadir comportamiento sin modificar el contrato; alternativa lógica transversal repetida.	Podría envolver un servicio con auditoría o control adicional. Debe preservar interfaz y evitar doble registro.
Strategy comportamiento	Elegir política de validación de documentos; alternativa condicional actual.	Una interfaz y estrategias por tipo hacen sustituible la validación si las reglas realmente difieren.

Conclusión del control: el mínimo por categorías no queda acreditado con la evidencia revisada. Debe confirmarse el criterio del docente y aportarse código, diagrama, justificación y pruebas de los patrones que finalmente se entreguen. No se recomienda introducir patrones sin una necesidad concreta de diseño.

4 Control actualizado de objetivos funcionales

Objetivo del enunciado	Evidencia estática actual	Límite pendiente
Convocatorias	Controller, servicio y consultas Angular	No hay ruta dedicada de administración verificada.
Registro de estudiantes	AuthService y pantalla registro	Ejecutar registro y validaciones.
Solicitudes	Servicio y pantallas estudiantil y administrativa	Verificar duplicidad y cancelación en SQL real.

Objetivo del enunciado	Evidencia estática actual	Límite pendiente
Documentación	Servicio, pantalla y almacenamiento	Probar límites y protección de descarga.
Evaluaciones	Servicios y pantalla evaluaciones	Probar asignación, puntajes y permisos.
Comités	Servicio y pantalla comites	Probar mínimo de miembros y reparto.
Aprobar o rechazar	ResolverAsync y solicitudes administrativas	Probar ambas rutas por monto.
Consultar estado	Vistas de solicitudes y notificaciones	Contrastar interfaz y persistencia.
Reportes básicos	ReporteService y ReportesController	No hay ruta dedicada de reportes verificada.

Se retira el porcentaje histórico de cinco de nueve objetivos completos: el documento de control era un corte antiguo y la presencia de nuevos componentes no equivale a aceptación funcional. La gestión de perfiles tiene pantalla de usuarios para coordinación; una pantalla individual de perfil no se acredita en app.routes.ts.

Anexo 9 Manual técnico

1 Propósito y prerrequisitos

Guía para reproducir el entorno académico. Los comandos siguientes se derivan de la estructura revisada y deben validarse en el equipo donde se ejecuten. No son un registro de instalación realizada en esta revisión.

Se necesitan Git, SQL Server accesible, SDK y runtime compatibles con API net8.0 y pruebas net9.0, además de Node.js y npm compatibles con Angular 18.2 y el lockfile. No se certifican requisitos mínimos de RAM o disco: los valores de los documentos anteriores no estaban sustentados por mediciones. No instalar automáticamente la versión más reciente de Angular CLI.

2 Obtener el código

```
git clone https://github.com/karencz0909/Proyecto_individual-_AnalisisSistemas.git
```

```
cd Proyecto_individual-_AnalisisSistemas
```

Registrar git rev-parse HEAD y git status antes de probar. La referencia de esta documentación es el commit indicado en la presentación. Los ZIP del repositorio no reemplazan la identificación de la versión de los directorios fuente.

3 Configurar SQL Server

Crear una base de prueba aislada y configurar ConnectionStrings:BecaNetDb para una instancia accesible. Program.cs usa EnsureCreated y luego ejecuta BecaNet_Nuevos_Requisitos.sql buscándolo en la raíz de la API o en la carpeta superior. El arranque se detiene si no logra preparar el esquema.

El script inicial y el modelo EF muestran diferencias entre tablas y vistas de perfiles. No ejecutar ambas vías indistintamente sobre una base con datos. Primero comprobar en una base desechable el esquema resultante, alta y lectura de los tres roles y ejecución del parche. Las migraciones EF no se prescriben como alternativa sin verificarlas. Conservar respaldo antes de cambiar un esquema existente.

4 Configurar y ejecutar la API

```
cd BecaNet-Backend/BecaNet.Api
dotnet restore
dotnet build
```

Configurar las claves Jwt:Secret o Jwt:Key, Jwt:Issuer y Jwt:Audience conforme a Program.cs y JwtService. El manual anterior mencionaba Jwt:Clave; esa clave no es la que lee la configuración actual. Usar un secreto propio y configuración externa sin publicarlo. Confirmar que los valores coinciden entre generación y validación del token.

Ejemplo para PowerShell en Windows, después de configurar la conexión y JWT:

```
$env:ASPNETCORE_ENVIRONMENT="Development"
$env:ASPNETCORE_URLS="http://localhost:5000"
dotnet run
```

La URL propuesta coincide con environment.ts. La terminal confirma el enlace real. En Development, Swagger se consulta en http://localhost:5000/swagger. Fuera de Development no se expone por la configuración actual. El puerto 7080 del manual anterior era un ejemplo y no coincide con el frontend revisado.

5 Instalar y ejecutar Angular

En otra terminal, desde la raíz del repositorio:

```
cd becanet-frontend
npm ci
npm start
```

npm ci utiliza el package-lock.json versionado. Si no se dispone de lockfile válido, revisar el estado antes de cambiar dependencias; npm install puede modificarlo. environment.ts apunta a http://localhost:5000/api. Abrir http://localhost:4200 y comprobar que la API y CORS aceptan ese origen. El cliente y la API deben permanecer activos.

6 Ejecutar pruebas y guardar evidencia

Desde la raíz, localizar el proyecto de pruebas explícitamente para evitar ejecutar dotnet test en un directorio sin proyecto o solución:

```
dotnet test BecaNet-Backend/BecaNet.Api.Tests/BecaNet.Api.Tests.csproj --logger trx
```

El proyecto de pruebas apunta a net9.0 y la API a net8.0; disponer de las herramientas y runtimes requeridos o alinear los destinos como cambio de código verificado. Se deben registrar restauración, descubrimiento de NUnit y xUnit y resultados completos. Esta tarea no modificó esas referencias.

Desde becanet-frontend, con un navegador de pruebas instalado:

```
npm test -- --watch=false --browsers=ChromeHeadless
```

El comando requiere que el ejecutable de Chrome esté disponible. Si falla, conservar el error y configurar el entorno; no marcar la suite como aprobada. Las pruebas con InMemory deben complementarse con SQL Server real para comprobar el esquema y los índices.

7 Estructura y endpoints de referencia

Ubicación	Contenido
BecaNet-Backend/BecaNet.Api	Controllers, Services, Repositories, DTOs, Models, Data, Security, Observers y Program.cs.
BecaNet-Backend/BecaNet.Api.Tests	Pruebas de solicitudes, documentos, comités, evaluaciones y asignaciones, además de archivos auxiliares.
becanet-frontend/src/app	features, pages, services, models, guard e interceptor de autenticación.
docs	Diagramas draw.io y un Anexo 1 previo.
wwwroot/uploads	Archivos cargados, organizados por identificador de solicitud.

Operación	Ruta de referencia	Acceso observado
Login	POST /api/Auth/login	Pública para autenticación
Mis solicitudes	GET /api/Solicitudes/mias	ESTUDIANTE

Operación	Ruta de referencia	Acceso observado
Resolver	PUT /api/Solicitudes/{id}/resolver	COORDINADOR
Mis asignaciones	GET /api/Asignaciones/mias	EVALUADOR
Auditoría	GET /api/auditoria	COORDINADOR
Mis notificaciones	GET /api/notificaciones/mias	ESTUDIANTE
Reportes	GET /api/Reportes/estadisticas-generales	Sin Authorize explícito observado

8 Solución de problemas

Síntoma	Verificación sugerida
CS0246	Revisar archivo, extensión cs, namespace, using y referencia del proyecto.
MSB1003	Ejecutar comando con ruta csproj explícita o ubicarse donde existe la solución.
API no inicia	Leer el error de conexión, permisos SQL y disponibilidad del parche. No silenciar una preparación fallida.
Angular no conecta	Comparar apiUrl, puerto de la API y origen CORS; revisar consola y red del navegador.
401 o 403	Comprobar token, expiración, issuer, audience y rol; no desactivar autorización para resolverlo.
Pruebas no se descubren	Comprobar SDK, target net9.0 y adaptadores NUnit y xUnit.
Archivo no se visualiza	Comprobar ruta, existencia y permisos. Evitar resolverlo haciendo públicos documentos privados.

9 Seguridad y futura publicación

La revisión encontró hash PBKDF2 con sal de 16 bytes, 100,000 iteraciones, SHA-256 y salida de 32 bytes, además de comparación de tiempo constante. Esto describe el algoritmo usado y no certifica toda la seguridad del producto.

Program.cs crea datos y cuentas predeterminadas y contiene valores JWT de desarrollo. Sustituirlos y condicionar el sembrado por entorno. Usar HTTPS, restringir CORS y proteger conexión y secretos. La presencia de guards Angular no protege por sí sola la API: varios controladores no tienen Authorize explícito y no se observó una política global que lo sustituya.

UseStaticFiles se ejecuta antes de autenticación y sirve wwwroot/uploads. Debe revisarse la descarga de expedientes mediante un endpoint autorizado y almacenamiento no público. La extensión y el tamaño no bastan para validar contenido seguro. No se publica ninguna contraseña de prueba en este manual.

Definir respaldo y recuperación conjunta de SQL y uploads y ensayar su restauración antes de usar datos reales. No se acredita infraestructura productiva, monitorización o un plan de recuperación ejecutado.

Anexo 10 Guía de uso por rol

Guía funcional basada en rutas y servicios observados. No sustituye un manual de usuario con capturas de una sesión ejecutada; esas capturas no se aportaron. Los nombres de campos pueden variar con el componente y deben contrastarse al realizar la demostración.

1 Acceso común

Abrir la aplicación local. Para registrarse como estudiante, usar /registro y completar los campos solicitados. Iniciar sesión en /login. El sistema debe mostrar las opciones del rol de la cuenta. No compartir credenciales entre roles; para probar doble coordinación se necesitan dos identidades diferentes.

2 Estudiante

Consultar las oportunidades abiertas visibles en la interfaz. Desde /mi-solicitud, seleccionar convocatoria y registrar motivo y monto. En /documentacion, elegir la solicitud correspondiente y adjuntar PDF, JPG, JPEG o PNG dentro del límite de 5 MB. Identificar la certificación con su tipo documental para que el expediente sea elegible en la distribución.

Consultar el estado y las notificaciones. Cancelar solo una solicitud propia mientras su convocatoria esté abierta. Si el sistema impide volver a postular después de cancelar, registrar el caso y contrastarlo con la regla de duplicidad e índice documentada. No asumir que una preaprobación equivale a beca concedida.

3 Evaluador

Entrar en /evaluaciones con la cuenta asignada. Consultar expedientes propios, revisar documentación y registrar puntaje de 0 a 100 con observaciones. Completar la revisión. Esta acción entrega la evaluación a coordinación; no aprueba la beca ni sustituye una de las dos decisiones requeridas por montos mayores.

4 Coordinador

En /usuarios administrar los perfiles disponibles. En /comites organizar integrantes. En /solicitudes-admin revisar el listado, asignar o distribuir expedientes y registrar decisiones conforme al estado y al monto. Usar /auditoria para consultar operaciones. Las funciones de administración de convocatorias y reportes tienen endpoints, pero no se identificó una ruta dedicada de interfaz en app.routes.ts; no se inventa un menú para ellas.

Para una solicitud de hasta Q10,000, resolver una vez evaluada. Para una mayor, confirmar revisión completada, registrar preaprobación y dejar que otro coordinador tome la decisión final. Al rechazar, consignar el motivo. La solicitud sigue pendiente durante la preaprobación favorable.

5 Mensajes y soporte de incidencias

Ante un error, guardar la operación, fecha, rol, identificador del expediente y mensaje visible, sin exponer token o contraseña. Evitar repetir decisiones finales para intentar corregir una pantalla. Confirmar el estado persistido y registrar el defecto para revisión.

6 Evidencias de uso pendientes

Completar capturas de login, postulación, documentación, asignación, evaluación, preaprobación, decisión final y auditoría. Cada captura debe indicar versión y contexto, usar datos ficticios y corresponder al código entregado. No se reutilizan diagramas antiguos con PostgreSQL como evidencia del motor actual SQL Server.

Fuentes y pendientes de cierre

Fuentes utilizadas

ID	Fuente	Uso
F01	proyecto 2026.pdf, 24 páginas	Enunciado, mínimos, rol docente y estructura de anexos 1 a 7.
F02	ANEXO_1_Karen.docx	Autora, carné, sede, correo, metas y cronograma histórico.
F03	Anexo 4 (1).docx	Visión, 6 épicas, 17 historias y criterios de aceptación.
F04	Anexo5_Documento_Diseno_BecaNet (1).docx	Diseño inicial contrastado y actualizado.
F05	Anexo 6 - Plan de Aseguramiento de la Calidad - BecaNet (1).docx	Base del plan de calidad, adaptado a modalidad individual.
F06	Anexo 7 - Plan de Pruebas Ágil - BecaNet (2).docx	Base del plan, ampliado con casos y límites.
F07	Control_Patrones_y_Objetivos_BecaNet (1).docx	Control histórico sustituido por evaluación con límites explícitos.
F08	Manual_Tecnico_BecaNet (1).docx	Guía inicial; corregidas rutas, JWT, pruebas y esquema.
F09	Repositorio GitHub y commit identificado	Código fuente, rutas, configuración, reglas y limitaciones.
F10	Resumen integral proporcionado por la autora	Contexto y funcionalidad contrastados con los archivos.

Pendientes que requieren información o validación

Prioridad	Pendiente	Dato o evidencia requerida
Alta	Aceptación de modalidad individual	Confirmar con docente adaptación de revisión entre pares, roles y ceremonias.
Alta	Patrones exigidos	Código y justificación que acrediten dos por categoría según criterio del curso.
Alta	Autorización y archivos	Corregir o verificar endpoints sin protección y descarga pública de uploads.
Alta	Esquema SQL y EF	Prueba de creación, actualización y CRUD sobre una base SQL Server aislada.

Prioridad	Pendiente	Dato o evidencia requerida
Alta	Resultados de pruebas	Salida fechada, commit, entorno y detalle de fallos o aprobaciones.
Media	Sprints y reuniones	Número, fechas, backlog comprometido, feedback y aceptación reales.
Media	Interfaces faltantes o integradas	Demostración verificable de administración de convocatorias, reportes y perfil propio.
Media	Reglas ambiguas	Confirmar repostulación tras cancelación y objetivos medibles no funcionales.
Media	Canal de comunicación	Confirmar canal con docente; teléfono solo si se requiere en el formato.
Media	Manual ilustrado	Capturas reales del recorrido con datos ficticios y versión identificada.

Control de cambios de la consolidación

Se centralizaron las fuentes en una sola portada y secuencia de anexos; se restablecieron los apartados del Anexo 1; se incorporaron los anexos 2 y 3 sin inventar actas; se conservó el backlog completo; se redibujaron vistas de diseño; se adaptaron calidad y pruebas a una única autora; se actualizaron el control de patrones y objetivos y los manuales. No se modificó el código, no se publicaron cambios ni se certificó aceptación académica.

La entrega documental permite revisar BecaNet de forma ordenada. La aceptación final depende de completar o reconocer los pendientes anteriores y aportar la evidencia correspondiente.